authorization MQTT/TLS/JSON

Interface Design Description

Service ID: *"authorization"*

Abstract
This document describes the MQTT protocol with TLS payload security and JSON payload encoding variant of the `authorization` service of the ConsumerAuthorization system.

Contents

1 Overview **3**

2 Service Operations **4**
2.1 operation grant . . . 4
2.2 operation revoke . . . 5
2.3 operation lookup . . . 5
2.4 operation verify . . . 5

3 Data Models **7**
3.1 struct MqttRequestTemplate . . . 7
3.2 struct MqttResponseTemplate . . . 7
3.3 struct ErrorMessage . . . 7
3.4 struct AuthorizationGrantRequest . . . 7
3.5 struct AuthorizationLookupRequest . . . 8
3.6 struct AuthorizationPolicy . . . 8
3.7 struct AuthorizationPolicyListResponse . . . 8
3.8 struct AuthorizationPolicyRequest . . . 8
3.9 struct AuthorizationPolicyResponse . . . 9
3.10 struct AuthorizationVerifyRequest . . . 9
3.11 struct MetadataRequirement . . . 9
3.12 Primitives . . . 10

4 References **11**

5 Revision History **12**
5.1 Amendments . . . 12
5.2 Quality Assurance . . . 12

1 Overview

This document describes the `authorization` service interface of the ConsumerAuthorization system over MQTT, which provides provider-level authorization policies: a provider declares which consumers may use its service (or event type), and any system can ask whether a consumer is authorized.

It's implemented using protocol, encoding as stated in below table:

Profile type	Type	Version
Transfer protocol	MQTT	3.1.1 [1]
Data encryption	TLS	1.3 [2], 1.2 [3]
Encoding	JSON	RFC 8259 [4]
Compression	N/A	-
Semantics	N/A	-
Ontology	N/A	-

Table 1: Communication and semantics details used for the `authorization` service interface

This document provides the Interface Design Description IDD to the *authorization – Service Description* document [5]. The operations, their semantics and the data models are the same as in the HTTP/TLS/JSON IDD [6]; this document specifies the MQTT binding.

The MQTT interface is offered only when the system runs with `mqtt.api.enabled=true`; it is then registered with the interface template `generic_mqtts` (`generic_mqtt` without TLS), the broker address and port (`mqtt.broker.address`, `mqtt.broker.port`) and the base topic `arrowhead/consumer-authorization/authorization/`. In the Arrowhead-test-cloud-java deployment the MQTT interfaces of the core systems are disabled.

2 Service Operations

The provider subscribes to one topic per operation, the base topic followed by the operation name:

Operation	Request topic
`grant`	`arrowhead/consumer-authorization/authorization/grant`
`revoke`	`arrowhead/consumer-authorization/authorization/revoke`
`lookup`	`arrowhead/consumer-authorization/authorization/lookup`
`verify`	`arrowhead/consumer-authorization/authorization/verify`

A consumer publishes a request envelope (MqttRequestTemplate) to the topic and receives the answer (MqttResponseTemplate) on its own `responseTopic`. The `authentication` field carries the requester's identity according to the provider's authentication policy: `IDENTITY-TOKEN//<token>` (outsourced or internal policy), `SYSTEM//<name>` (declared policy) or the requester's X.509 certificate (certificate policy). The `status` of the response uses the HTTP status code numbers (200, 201, 204, 400, 401, 403, 404, 408, 423, 500, 503); on error the payload is an ErrorMessage.

```
{
  "traceId": "a1b2c3",
  "authentication": "IDENTITY-TOKEN//<token>",
  "responseTopic": "arrowhead/response/TemperatureDisplay/a1b2c3",
  "qosRequirement": 1,
  "params": {},
  "payload": {}
}
```

Listing 1: Request envelope.

```
{
  "status": 200,
  "traceId": "a1b2c3",
  "receiver": "TemperatureDisplay",
  "payload": {}
}
```

Listing 2: Response envelope.

2.1 PUBLISH `arrowhead/consumer-authorization/authorization/grant`

Operation: grant
Input: AuthorizationGrantRequest
Output: AuthorizationPolicyResponse

Creates or replaces the provider-level policy of the requesting provider for one target. Returns 201 for a new policy, 200 if an identical one exists.

```
{
  "cloud": "LOCAL",
  "targetType": "SERVICE_DEF",
  "target": "temperatureReading",
  "description": "Displays may read the temperature",
  "defaultPolicy": {
    "policyType": "WHITELIST",
    "policyList": [
      "TemperatureDisplay"
    ],
    "policyMetadataRequirement": {
      "unit": "celsius"
```

```
    }
  },
  "scopedPolicies": {
    "read": {
      "policyType": "ALL"
    }
  }
}
```

Listing 3: Payload of an grant request.

2.2 PUBLISH arrowhead/consumer-authorization/authorization/revoke

Operation: revoke
Input: String

Removes a provider-level policy of the requester by instance identifier. Returns 200 if removed, 204 if it did not exist.

```
"TemperatureSensor|temperatureReading|1.0.0"
```

Listing 4: Payload of an revoke request.

2.3 PUBLISH arrowhead/consumer-authorization/authorization/lookup

Operation: lookup
Input: AuthorizationLookupRequest
Output: AuthorizationPolicyListResponse

Returns the requester's provider-level policies matching the filter.

```
{
  "instanceIds": [
    "TemperatureSensor|temperatureReading|1.0.0"
  ],
  "cloudIdentifiers": [
    "string"
  ],
  "targetNames": [
    "string"
  ],
  "targetType": "SERVICE_DEF"
}
```

Listing 5: Payload of an lookup request.

2.4 PUBLISH arrowhead/consumer-authorization/authorization/verify

Operation: verify
Input: AuthorizationVerifyRequest
Output: Boolean

Evaluates whether the consumer may use the target (and operation) of the provider: management-level policies first, provider-level policies if none applies; no applicable policy means denied. Returns a boolean.

```
{
  "provider": "TemperatureSensor",
  "consumer": "TemperatureDisplay",
  "cloud": "LOCAL",
```

```
  "targetType": "SERVICE_DEF",
  "target": "temperatureReading",
  "scope": "read"
}
```

Listing 6: Payload of an verify request.

3 Data Models

Here, all data objects that can be part of the service calls associated with this service are listed in alphabetic order. Each *struct* denotes a JSON Object with the named fields; fields that are not needed may be omitted.

3.1 struct MqttRequestTemplate

The request envelope.

Object Field	Value Type	Description
"traceId"	String	Identifier chosen by the requester, returned in the response.
"authentication"	String	Identity of the requester (see Section 2).
"responseTopic"	String	Topic on which the response is published.
"qosRequirement"	Number	MQTT QoS of the response (0, 1 or 2).
"params"	Object<String>	Optional parameters (the query parameters of the HTTP binding).
"payload"	Object	The operation input.

3.2 struct MqttResponseTemplate

The response envelope.

Object Field	Value Type	Description
"status"	Number	Status code.
"traceId"	String	The trace identifier of the request.
"receiver"	String	Name of the requester.
"payload"	Object	The operation output, or an ErrorMessage.

3.3 struct ErrorMessage

Object Field	Value Type	Description
"errorMessage"	String	
"errorCode"	Number	
"exceptionType"	ExceptionType	
"origin"	String	

3.4 struct AuthorizationGrantRequest

A provider-level authorization policy that the provider sets for its own service.

Object Field	Value Type	Description
"cloud"	String	Cloud identifier of the consumer cloud the policy applies to; `LOCAL` for the own cloud.
"targetType"	String	Target type: `SERVICE_DEF` or `EVENT_TYPE`.
"target"	String	Service definition (or event type) the policy protects.
"description"	String	Free-text description.
"defaultPolicy"	AuthorizationPolicyRequest	Policy that applies to all operations without a scoped policy.
"scopedPolicies"	Object<AuthorizationPolicyRequest>	Policies for individual operations, keyed by operation name.

3.5 struct AuthorizationLookupRequest

Filter for the requester's own provider-level policies.

Object Field	Value Type	Description
"instanceIds"	Array<String>	Policy instance identifiers to match.
"cloudIdentifiers"	Array<String>	Cloud identifiers to match.
"targetNames"	Array<String>	Target names to match.
"targetType"	String	Target type: `SERVICE_DEF` or `EVENT_TYPE`.

3.6 struct AuthorizationPolicy

One policy rule.

Object Field	Value Type	Description
"policyType"	AuthorizationPolicyType	Policy type: `ALL`, `WHITELIST`, `BLACKLIST` or `SYS_METADATA`.
"policyList"	Array<String>	System names of the white- or blacklist.
"policyMetadata Requirement"	MetadataRequirement	Metadata the consumer system must have (`SYS_METADATA`).

3.7 struct AuthorizationPolicyListResponse

List of authorization policies.

Object Field	Value Type	Description
"entries"	Array<AuthorizationPolicy Response>	Matching records of the current page.
"count"	Number	Total number of matching records (all pages).

3.8 struct AuthorizationPolicyRequest

One policy rule.

Object Field	Value Type	Description
"policyType"	String	Policy type: `ALL`, `WHITELIST`, `BLACKLIST` or `SYS_METADATA`.
"policyList"	Array<String>	System names of the white- or blacklist.
"policyMetadata Requirement"	MetadataRequirement	Metadata the consumer system must have (`SYS_METADATA`).

3.9 struct AuthorizationPolicyResponse

A stored authorization policy.

Object Field	Value Type	Description
"instanceId"	String	Policy instance identifier `<MGMT or PR>|<cloud>|<provider>|<targetType>|<target>`.
"level"	AuthorizationLevel	Policy level: `MGMT` (set by an operator) or `PROVIDER` (set by the provider itself).
"cloud"	String	Cloud identifier of the consumer cloud the policy applies to; `LOCAL` for the own cloud.
"provider"	String	Provider system the policy protects.
"targetType"	AuthorizationTargetType	Target type: `SERVICE_DEF` or `EVENT_TYPE`.
"target"	String	Target of the policy or token: a service definition or event type.
"description"	String	Free-text description.
"defaultPolicy"	AuthorizationPolicy	Policy that applies to all operations without a scoped policy.
"scopedPolicies"	Object<AuthorizationPolicy>	Policies for individual operations, keyed by operation name.
"createdBy"	String	Name of the system that created the record.
"createdAt"	DateTime	UTC time of creation.

3.10 struct AuthorizationVerifyRequest

One authorization question: may the consumer use the target (operation) of the provider?

Object Field	Value Type	Description
"provider"	String	Provider system; for `verify` defaults to the requester.
"consumer"	String	Consumer system; for `verify` defaults to the requester.
"cloud"	String	Cloud of the consumer, `LOCAL` by default.
"targetType"	String	Target type: `SERVICE_DEF` or `EVENT_TYPE`.
"target"	String	Target of the policy or token: a service definition or event type.
"scope"	String	Operation to check; empty checks the default policy.

3.11 struct MetadataRequirement

A JSON object of metadata requirements: each key is a metadata path (dot notation), each value the required value or an operator object such as `{"op": "LESS_THAN", "value": 10}`.

3.12 Primitives

As all messages are encoded using the JSON format [4], the following primitive constructs, part of that standard, become available.

Type	Description
Array	A JSON Array [4]; Array<A> has elements of type A.
AuthorizationLevel	One of `MGMT`, `PROVIDER` (JSON String).
AuthorizationPolicyType	One of `ALL`, `WHITELIST`, `BLACKLIST`, `SYS_METADATA` (JSON String).
AuthorizationTargetType	One of `SERVICE_DEF`, `EVENT_TYPE` (JSON String).
Boolean	JSON Boolean, `true` or `false`.
DateTime	JSON String with an ISO 8601 UTC timestamp, e.g. `2026-10-22T12:00:00Z`.
ExceptionType	One of `ARROWHEAD`, `INVALID_PARAMETER`, `AUTH`, `FORBIDDEN`, `DATA_NOT_FOUND`, `TIMEOUT`, `LOCKED`, `INTERNAL_SERVER_ERROR`, `EXTERNAL_SERVER_ERROR` (JSON String).
Number	JSON Number, an integer.
Object	JSON Object; Object<A> has string keys and values of type A.
String	JSON String.

4 References

[1] A. Banks and R. Gupta, “Mqtt version 3.1.1,” OASIS, Standard, 2014.

[2] E. Rescorla, “The transport layer security (tls) protocol version 1.3,” IETF, RFC 8446, 2018.

[3] T. Dierks and E. Rescorla, “The transport layer security (tls) protocol version 1.2,” IETF, RFC 5246, 2008.

[4] T. Bray, “The javascript object notation (json) data interchange format,” IETF, RFC 8259, 2017.

[5] J. Delsing, “authorization – Service Description (ConsumerAuthorization),” Eclipse Arrowhead, SD 5.2.2, 2026, consumerAuthorization-v5.2.2-authorization-SD.pdf.

[6] ——, “authorization HTTP/TLS/JSON – Interface Design Description (ConsumerAuthorization),” Eclipse Arrowhead, IDD 5.2.2, 2026, consumerAuthorization-v5.2.2-authorization-HTTP-IDD.pdf.

5 Revision History

5.1 Amendments

No.	Date	Version	Subject of Amendments	Author
1	2026-10-22	5.2.2	Document created from the Arrowhead LaTeX templates for the 5.2.2 Java Spring implementation, including the local extensions of this workspace.	Jerker Delsing

5.2 Quality Assurance

No.	Date	Version	Approved by
1		5.2.2	